Appello 24 luglio 2017



Politecnico di Milano

Anno accademico 2016-2017

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Cugola	☐ Mottola	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta l'impossibilità a partecipare all'appello successivo.
6. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si consideri la seguente specifica di un'interfaccia `Deque`. Una `Deque` è una sequenza di elementi che consente inserimento ed eliminazione sia all'inizio che alla fine della sequenza. La specifica dell'interfaccia e' data nel seguito attraverso semplici commenti testuali.

```
public interface Deque<E> extends Collection<E>{
/*OVERVIEW: Collezione mutabile di oggetti, di tipo E,
 * organizzati in una sequenza x1, x2, ... xn
 * Rimozione e inserimento di elementi possono avvenire
 * solo in prima o ultima posizione.
 */

//inizializza this alla  Deque vuota;
public Deque()

//osservatori:

//restituisce il numero di elementi di this
public /*@ pure @*/ int size();


// restituisce, senza rimuoverlo, il primo elemento,
//ma lancia eccezione se this e' vuoto;
public /*@ pure @*/ E getFirst() throws NoSuchElementException;

// restituisce, senza rimuoverlo, l'ultimo elemento,
//ma lancia eccezione se this e' vuoto;
public /*@ pure @*/ E getLast() throws NoSuchElementException;

//mutators:

//aggiunge x in prima posizione, ma lancia eccezione se x e' null;
public void addFirst(E x) throws NullPointerException;

//aggiunge x in ultima posizione, ma lancia eccezione se x e' null;
public void addLast(E x) throws NullPointerException;

//elimina il primo elemento, ma lancia eccezione se this e' vuoto
public void removeFirst()  throws NoSuchElementException;

//elimina l'ultimo elemento, ma lancia eccezione se this e' vuoto
public void removeLast()  throws NoSuchElementException;


}
```

Si rammenta che l'interfaccia Collection prevede gli usuali metodi booleani `contains(Object o)`, `containsAll(Collection<?> c)` e inoltre il metodo `toArray()` qui di seguito specificato:

```
//restituisce un array che contiene tutti gli elementi di this
//nella sequenza dal primo all'ultimo
public /*@ pure @*/ Object[] toArray() .
```

Domanda a

Considerati come dati i metodi puri, specificare in JML i metodi `addLast, removeFirst`.

Soluzione

```
//@ensures x!=null && getLast()==x && size()==\old(size()+1) &&
```

```
//@(\forall int i; 0<=i && i<size()-1;
//@ this.toArray()[i]==\old(this.toArray()[i]));
//@signals (NullPointerException e) x==null;
public void addLast(E x) throws NullPointerException

//@ensures x!=null && getFirst()==this.toArray()[0] && size()==\old(size()-1) &&
//@(\forall int i; 0<=i && i<size();
//@ this.toArray()[i]==\old(this.toArray()[i+1]));
//@signals (NoSuchElementException e) \old(size()==0);
public void removeFirst() throws NoSuchElementException
```

Domanda b

Si consideri la seguente classe `ArrayDeque`, di cui nel seguito è descritta solo un parte. L'idea dell'implementazione è di avere due indici, `head` e `tail`, che rappresentano la posizione, rispettivamente, del primo e dell'ultimo elemento all'interno di un array `elements`. Gli indici si muovono in modo circolare, con `head` che viene decrementato quando si aggiunge un elemento in testa (e incrementato quando si rimuove) mentre `tail` è incrementato quando si aggiunge un elemento in coda (e decrementato quando si rimuove).

Il campo `head` rappresenta la posizione del primo elemento in coda. I successivi elementi sono "alla sua destra" (considerando la matematica in modulo). Il campo `tail` rappresenta invece la posizione del primo elemento vuoto alla fine della coda. I precedenti elementi sono "alla sua sinistra" (considerando la matematica in modulo). Quindi se `size()>0` allora `elements[head]` è il primo elemento, `elements[tail-1]` è l'ultimo.

Quando, dopo un inserimento in testa o in coda, `head` e `tail` diventano uguali, la coda è piena: la dimensione dell'array viene allora raddoppiata, tramite la chamata di un metodo privato `doubleCapacity()` che costruisce un array di capacità doppia, vi ricopia tutti gli elementi e poi pone *elements* uguale al nuovo array. Il metodo, al termine dell'esecuzione, garantisce che valga la condizione

```
head==0 && tail==\old(elements.length)
```

ossia che la `head` sia inizializzata a 0 e la `tail` sia portata al primo elemento libero del nuovo array.

```
public class ArrayDeque<E> implements Deque<E> {
// Overview: Implementazione basata su array flessibili dell'interfaccia Deque.
// Si usa un array circolare, con due indici, head e tail

  private E[] elements;
  private int head; // indice del primo elemento
  private int tail; // indice dell'ultimo elemento
  private int num;  // numero degli elementi in coda
  //costruttore
  public ArrayDeque() {
    elements = (E[]) new Object[16];
    head = tail = num = 0;
  }

  public void addFirst(E e) {
    if (e == null) throw new NullPointerException();
    head--;
    if (head<0) head = elements.length -1;
    elements[head] = e;
    if (head == tail) doubleCapacity();
    num++;
  }

  public void addLast(E e) {
    if (e == null) throw new NullPointerException();
    elements[tail] = e;
    tail = (tail + 1) % elements.length;
    if (tail == head) doubleCapacity();
    num++;
```

```
  }

  public E getFirst() {
    if (num == 0) throw new NoSuchElementException();
    return elements[head];
  }

  public E removeFirst() {
    if (num == 0) throw new NoSuchElementException();
    E result = elements[head];
    elements[head]=null;
    head = (head + 1) % elements.length;
    num--;
    return result;
  }

  public int size() {
    return num;
  }
  ...
}
```

Si descrivano l'invariante di rappresentazione e la funzione di astrazione della classe `ArrayDeque`, ad esempio tramite due opportuni private invariant.

Soluzione

L'invariante di rappresentazione specifica le condizioni sull'array `elements`, sul range tra `head` e `tail`, e infine specifica il numero di elementi `num` nei due casi: quando `head<tail` (gli elementi sono tutti compresi fra la posizione `head` e la posizione `tail` -1) e quando `head>tail` (gli elementi sono dalla posizione `head` alla fine dell'array, e fra 0 e la posizione `tail-1`). Inoltre, tutti gli elementi della coda sono diversi da null.

```
private invariant
elements!=null && 0<=tail && tail<elements.length &&
0<=head && head<elements.length &&
(head==tail <==> num==0) &&
(head<tail ==> num==tail-head) &&
(head>tail ==> num==(elements.length - head) + tail &&
(\forall i; 0<=i && i<num; elements[(head+i) % elements.length]!=null;
```

La funzione di astrazione può essere rappresentata semplicemente specificando quali sono gli elementi restituiti dal metodo `toArray` e quale e' la dimensione della Deque:

```
private invariant
   size()==num && this.toArray().length==num &&
   (\forall i; 0<=i && i<num; this.toArray[i] ==  (head+i) % elements.length;
```

o anche implementando (con analogo meccanismo) il metodo `toString`.

Esercizio 2

Si deve progettare un'applicazione di posta elettronica. Il programma mantiene l'insieme dei messaggi ricevuti dall'utente, organizzato in un insieme di cartelle. Ogni cartella ha un nome, un ordine di visualizzazione dei messaggi (per data, per titolo o per mittente) e contiene messaggi o altre cartelle. E' possibile aggiungere messaggi ad una cartella, rimuoverli, o spostarli in un'altra cartella. Ogni messaggio ha una serie di campi di informazione (mittente, titolo, lista dei destinatari, lista degli utenti che devono ricevere una copia –campo CC–, data di invio, codice identificativo), un testo e, opzionalmente, una serie di allegati. Ogni allegato può

essere un testo, un'immagine, un programma eseguibile o un generico allegato binario. Ogni allegato può essere mostrato oppure salvato su file. Mostrare un allegato vuol dire mostrarlo a video nel caso di testo o immagini, oppure eseguirlo nel caso si tratti di un programma. Gli allegati binari generici non possono essere mostrati a video. Un messaggio può essere mostrato (il che equivale a mostrare i campi di informazione ed il testo del messaggio), salvato su file, oppure ritrasmesso ad un altro insieme di utenti. Infine è possibile rispondere al messaggio.

Domanda A.

Si progetti, in UML, l'insieme di classi che descrivono la struttura dati di tale programma di posta.

Domanda B.

Si fornisca il diagramma delle sequenze per lo scenario che prevede l'arrivo di un nuovo messaggio con una immagine allegata, l'inserimento nella cartella `INBOX`, la visualizzazione del messaggio e del relativo allegato e il successivo spostamento nella cartella `Amici`.

Domanda C.

Si estenda il diagramma UML iniziale per considerare il caso in cui il programma di posta possa gestire anche messaggi SMS, con le relative caratteristiche e differenze rispetto ai messaggi di posta, supponendo però che anche gli SMS possano essere classificati dall'utente inserendoli in opportune cartelle (distinte da quelle dei messaggi di posta).

Soluzione

Esercizio 3

Si consideri la seguente classe Java:

```
public class DataStore {
    private List<Integer> data1, data2;
    public DataStore() {
        data1 = new ArrayList<Integer>();
        data2 = new ArrayList<Integer>();
    }
    public void add1(int elem) {
        synchronized(data1) { data1.add(elem); }
    }
    public void remove1(int elem) {
        synchronized(data1) { data1.remove(elem); }
    }
    public void add2(int elem) {
        synchronized(data2) { data2.add(elem); }
    }
    public void remove2(int elem) {
        synchronized(data2) { data2.remove(elem); }
    }
}
```

Domanda a

Le istanze della classe sono "thread safe", ovvero i metodi possono essere invocati da thread diversi senza che questo crei problemi? In caso negativo, come dobbiamo modificare il codice per garantire il corretto accesso alle istanze della classe da parte di thread diversi?

Soluzione

La sincronizzazione è corretta.

Domanda b

Si aggiunga il metodo `void copia12()` che copia il contenuto della lista `data1` nella lista `data2`. Al termine dell'esecuzione le due liste devono contenere gli stessi elementi. Si garantisca la corretta sincronizzazione del metodo in presenza di thread multipli.

Soluzione

```
public void copia12() {
    synchronized(data1) {
        synchronized(data2) {
            data2.clear();
            data2.addAll(data1);
        }
    }
}
```

Domanda c

Si aggiunga il metodo `void copia12Async()` che opera come il precedente metodo `copia12` ma esegue in un thread parallelo rispetto al chiamante.

Soluzione

```
public void copia12Async() {
    new Thread() {
        public void run() { copia12(); }
    }.start();
}
```

Esercizio 4

Si consideri il seguente metodo Java:

```
public int cercaSequenzaCrescentePiuLunga (int vett[]) {
  int i = 0,cont=0,max = 0;
  for (i = 1; i < vett.length; i++) {
    if (vett[i]>vett[i-1]) {
      cont++;
      if (cont > max) {
        max = cont;
      }
    } else {
      cont = 0;
    }
   }
 return max;
}
```

1. Si definisca il diagramma di flusso di controllo.
2. Si fornisca un insieme di cardinalità minima di casi di test per la copertura delle decisioni.
3. Quale altro criterio di test viene soddisfatto dai casi di test individuati al punto precedente?

Soluzione

E' sufficiente un singolo dato di test, costituito da un vettore di 4 elementi come ad esempio:

$$[0, 1, 0, 1]$$

per esercitare tutte le decisioni (branch). La lunghezza minima del vettore deve essere 4, in quanto il ciclo deve essere eseguito almeno 3 volte per coprire i tre branch al suo interno.

Ovviamente la copertura delle decisioni implica sempre anche quella delle istruzioni. In questo caso, e' soddisfatto anche il criterio di copertura delle condizioni (in quanto tutte le espressioni condizionali sono atomiche).